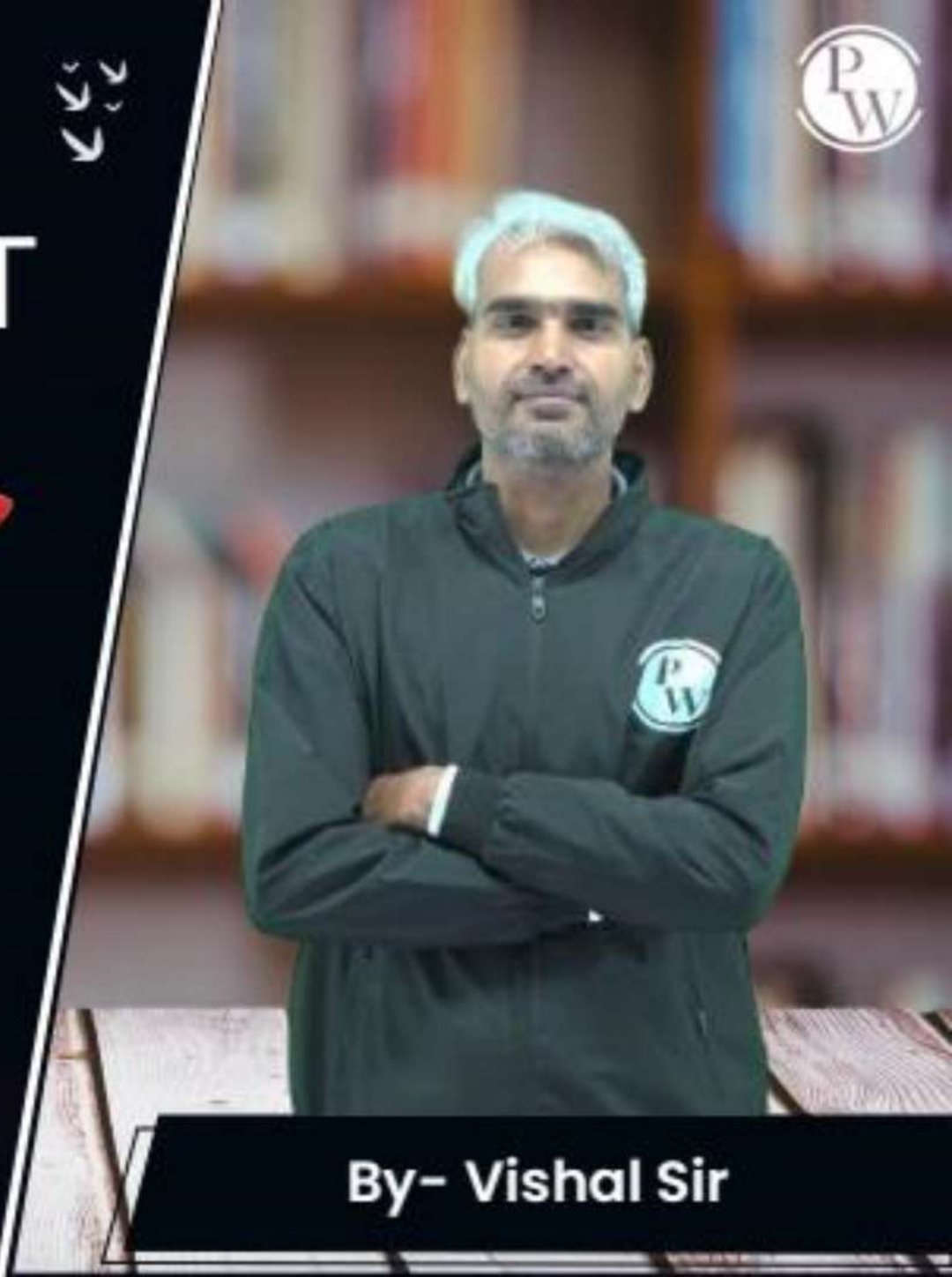
Computer Science & IT



Compiler Design

Intermediate Code Generation
&
Code Optimization

Lecture No. 03



By- Vishal Sir

Recap of Previous Lecture



Topic Intermediate Representation

Topic Representation of Three Address Codes

- Quadruples
- Triple
- Indirect Triple

Topic Code Optimization

Slide

Topics to be Covered

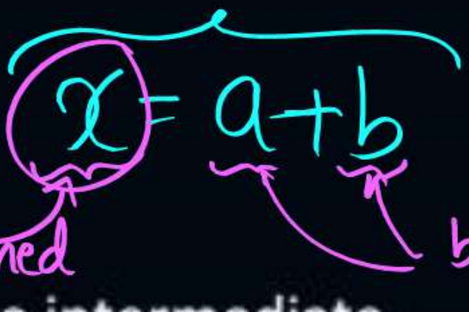


- Topic: Static Single Assignment (SSA)
- Topic: Control Flow Graph
- Topic: Liveness Analysis
- Topic: Miscellaneous

Slide



Topic : Static Single Assignment



- Static Single Assignment (SSA) is a means of structuring the intermediate representation such that every variable is allotted a value only once and every variable is defined before it's use

That is,

- A Variable can not be used more than once in the L.H.S.
- Every variable is defined before it is used.

E.g.



Slide

Ans = 8

Q1. The least number of temporary variables required to create a three-address code in static single assignment form for the expression

q + r / 3 + s − t * 5 + u * v / w is

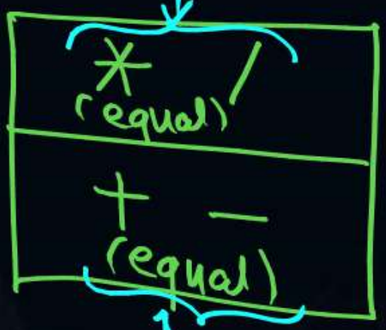
Ans = 8

$t_1 = r / 3$
$t_2 = t * 5$
$t_3 = u * v$
$t_4 = t_3 / w$
$t_5 = q + t_1$ =>
$t_6 = t_5 + s$
$t_7 = t_6 - t_2$
$t_8 = t_7 + t_4$

Total number of Variables in SSA Code = 7 + 8 = 15

7 : defined from outside
8 : defined inside

Left Associative



Left Associative

Slide

Q. Consider the following statement

$$x = a + b - b + c$$

Minimum number of variables required in 3AC	Minimum number of variable required in SSA form
$x = a + b - b + c$ $x = a + c$ ⇓ Three add^{r} Code using minimum variable $a = a + c$ Minimum two Variables in 3AC	$a_1 = a + c$ → Minimum Three Variable in SSA.

Q2. Consider the following intermediate program in three address code

```
p = a – b
q = p * c
p = u * v
q = p + q
```

which one of the following corresponds to a static single assignment form of the above code?

A

$$p_1 = a - b$$
$$q_1 = p_1 * c$$
$$p_1 = u * v$$
$$q_1 = p_1 + q_1$$

B

$$p_3 = a - b$$
$$q_4 = p_3 * c$$
$$p_4 = u * v$$
$$q_5 = p_4 + q_4$$

C

$$p_1 = a - b$$
$$q_1 = p_2 * c$$
$$p_3 = u * v$$
$$q_2 = p_4 + q_3$$

D

$$p_1 = a - b$$
$$q_1 = p * c$$
$$q_2 = u * v$$
$$q_2 = p + q$$

Slide

Q3. Consider the following code segment:

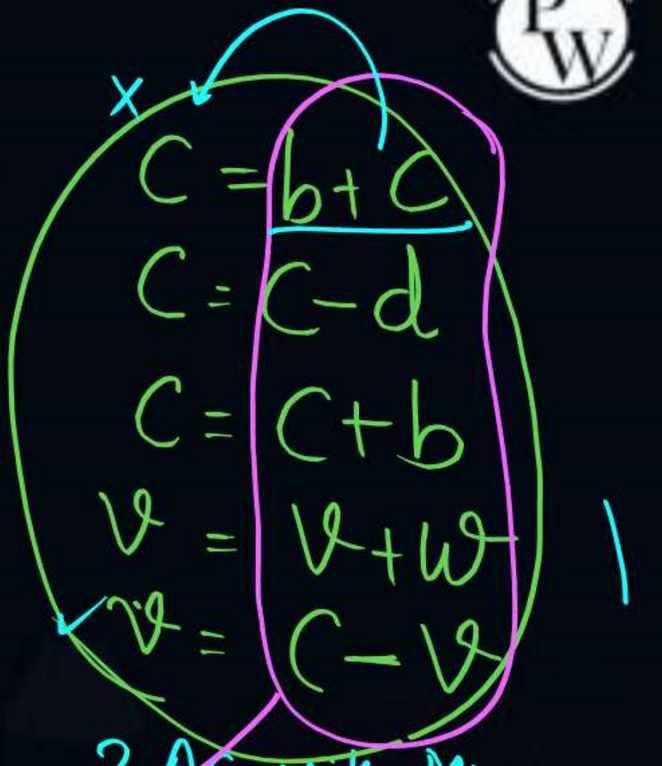
```
a = b + c
k = a - d
a = k + b
u = v + w
u = a - u
```

(a - u)

((k+b) - (v+w))

(((a-d)+b) - (v+w))

((((b+c)-d)+b) - (v+w))



3 AC with Min No. of Variables

∴ Min no. of Variables in SSA

10 = (b, c, d, v, w) + 5 statements
(5 Variables + 5 Variables)

Slide

Q. $x = (a*b) + a + (b*c)$

$c = b*c$

$b = a*b$

$b = b+a$

$b = b+c$

Minimum number of variable in 3 AC = 3

Minimum number of variables in SSA = 3 + 4

{b, c, a} + {4 statements}



Topic : Control Flow Graph

Control Flow graph
Can be used for
data flow analysis

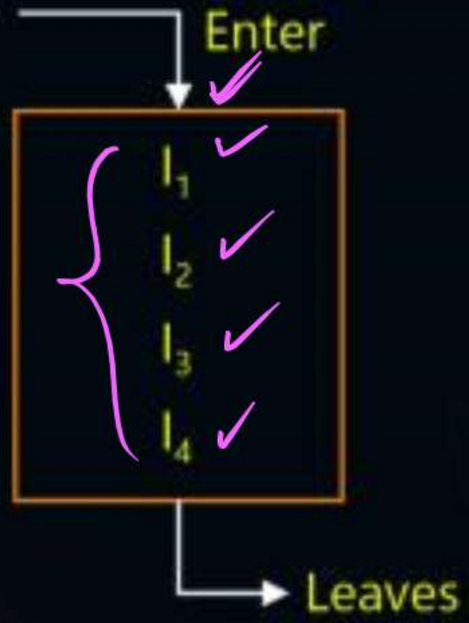
- Control Flow Graph comprises of nodes and edges.
- Control flow graph is collection of basic blocks and controls.

?



Topic : Basic Block

- Basic Blocks is a sequence of 3-address code where control enters at the beginning and leave only at the end.



Basic Blocks

- Basic block is a set of statements that always execute in a sequence one after other.



Topic : Basic Block

- Three Address code for the expression

$a = b + c * d$

$t_1 = c * d$

$t_2 = b + t_1$

$a = t_2$

Basic Block

- Here all Statements execute in a sequence one after other.



Topic : Basic Block

- Consider the following Three address code

```
(1) If A < B goto (4)
(2) t1=0
(3) goto(5)
(4) t1=1
(5) exit
```

- NOTE : The statement do not execute in a sequence one after other so it is not the basic block.

Slide



Topic : Finding the basic block

- In order to find the basic blocks we need to find the leader in the program. Then a basic block will start from one leader to next leader but not including the next leader.



Topic : Identifying leader in a Basic block

1. First statement is always a leader.
2. Statement that is target of conditional or unconditional statement is a leader.
3. Statement that follow immediately a conditional or unconditional statement is a leader.

Slide

Topic : Control Flow Graph

① leader

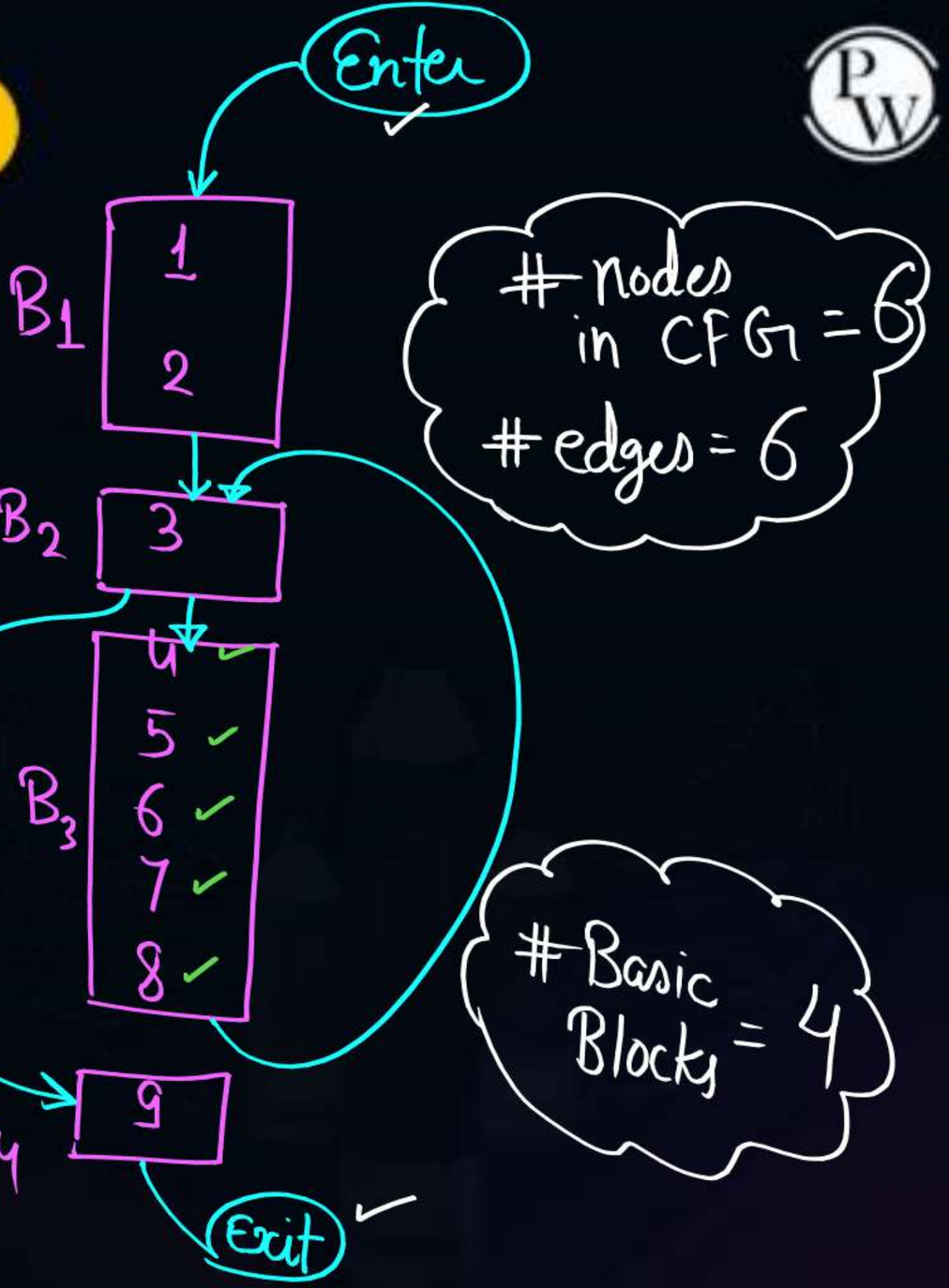
② leader

③ leader

```
1.  f = 1;
2.  i = 2;
3.  if (i > x) goto 9
4.  t1 = f * i ;
5.  f = t1;
6.  t2 = i + 1;
7.  i = t2 ;
8.  goto (3)
9.  Return f
```

③ ② leader

Control Flow graph



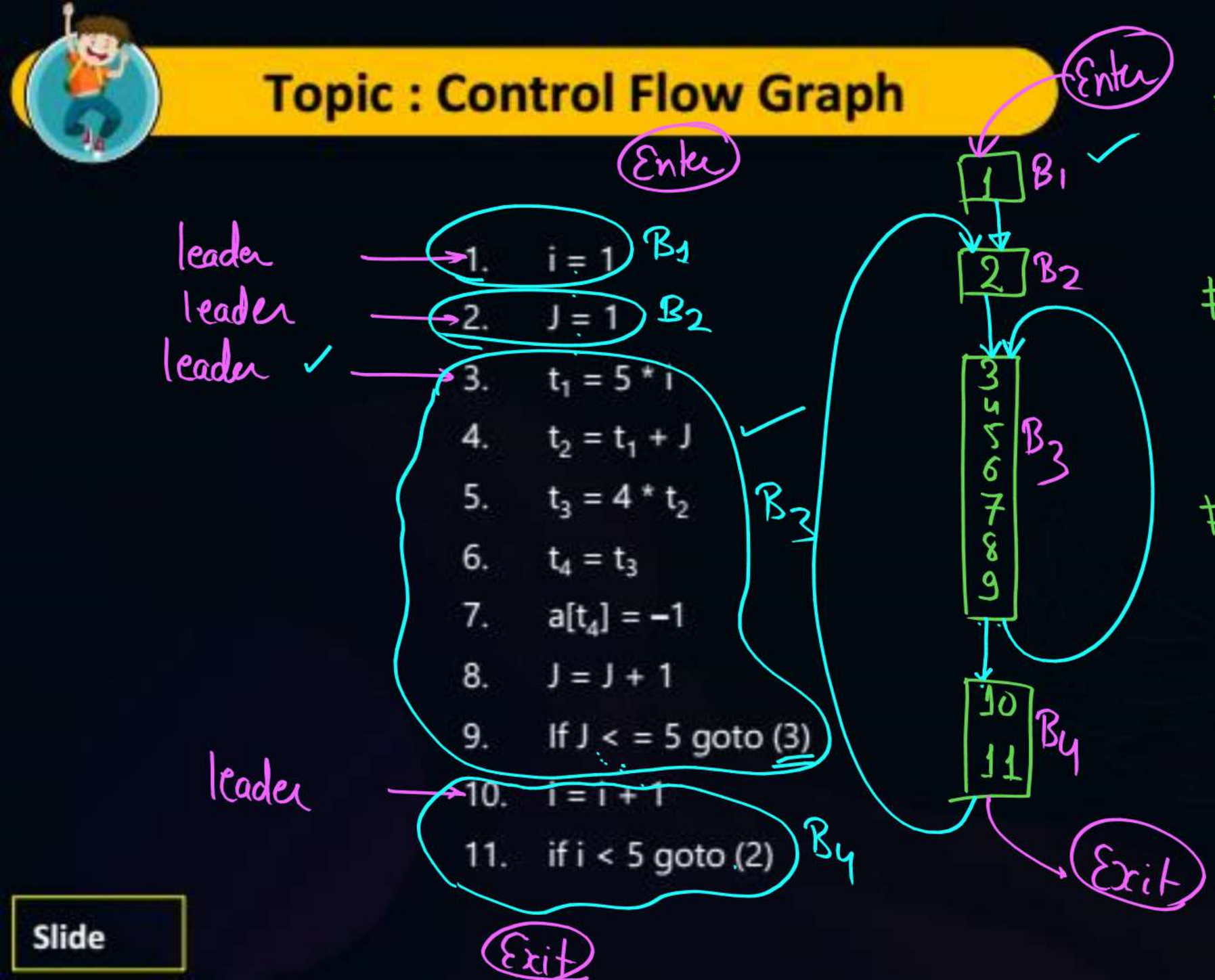
nodes in CFG = 6

edges = 6

Basic Blocks = 4

Slide

Topic : Control Flow Graph



Nodes in CFG = 6

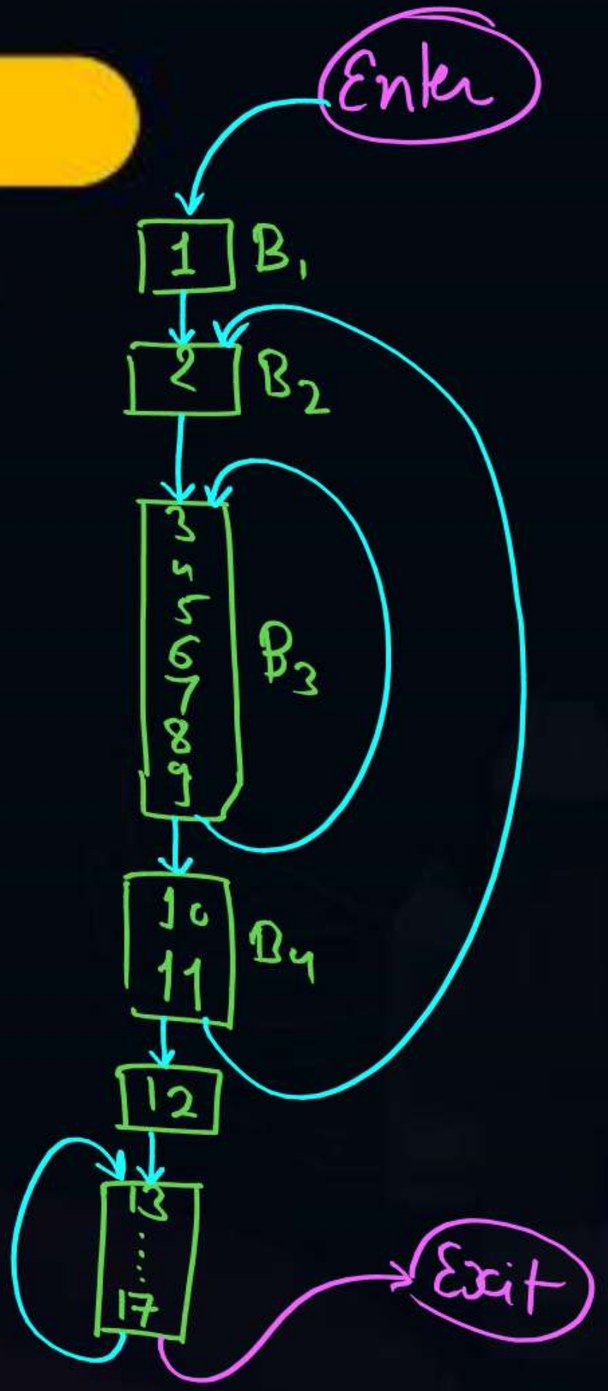
Edges in CFG = 7

Basic Blocks : ? 4

Slide

Topic : Control Flow Graph

leader → 1. i = 1
leader → 2. J = 1
leader → 3. $t_1 = 10 * i$
4. $t_2 = t_1 + J$
5. $t_3 = 8 * t_2$
6. $t_4 = t_3 - 88$
7. $a[t_4] = 0.0$
8. J = J + 1
9. If J < = 10 goto (3)
leader → 10. i = i + 1
11. if i <= 10 goto (2)
leader → 12. i = 1
leader → 13. $t_5 = i - 1$
14. $t_6 = 88 * t_5$
15. $a[t_6] = 1.0$
16. i = i + 1
17. if i < = 10 goto (13)



nodes = 8

Basic Blocks = 6

Edges =

Slide

Topic : Control Flow Graph

Leader → (1) PROD = 0

(2) I = 1

(3) T2 = addr(A) – 4

(4) T4 = addr(B) – 4

leader → (5) T1 = 4 × I

(6) T3 = T2[T1]

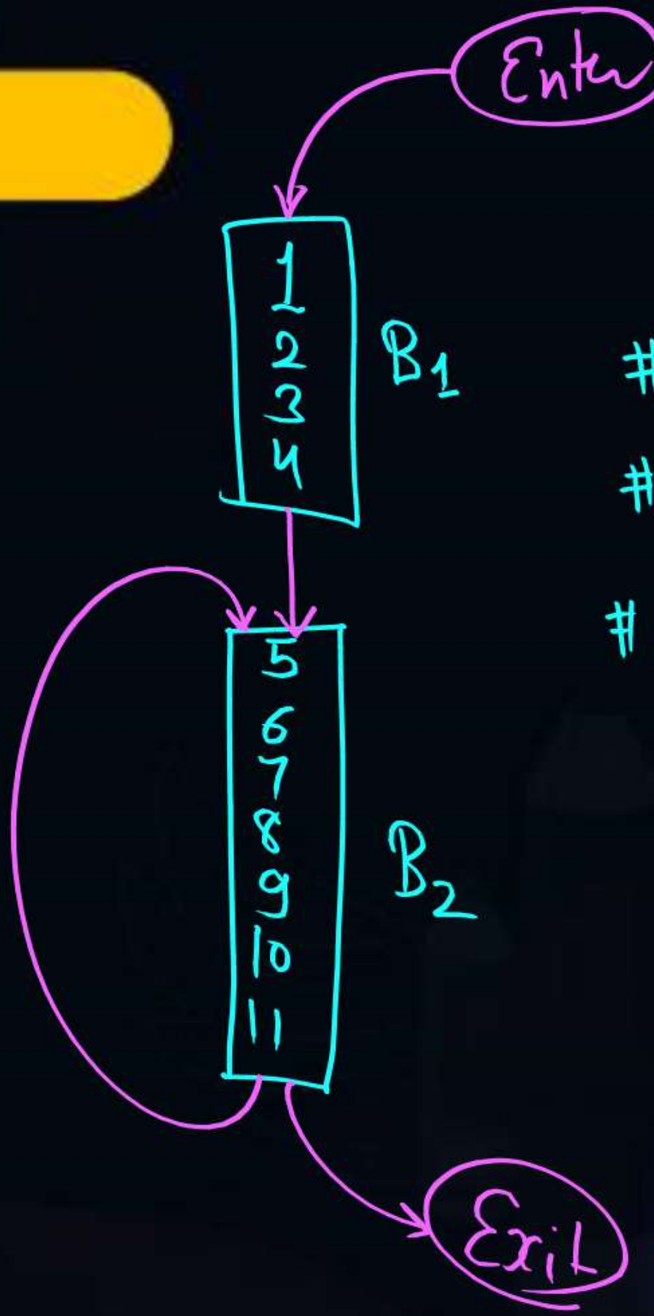
(7) T5 = T4[T1]

(8) T6 = T3 × T5

(9) PROD = PROD + T6

(10) I = I + 1

(11) IF I <=20 GOTO (5)



Nodes = 4

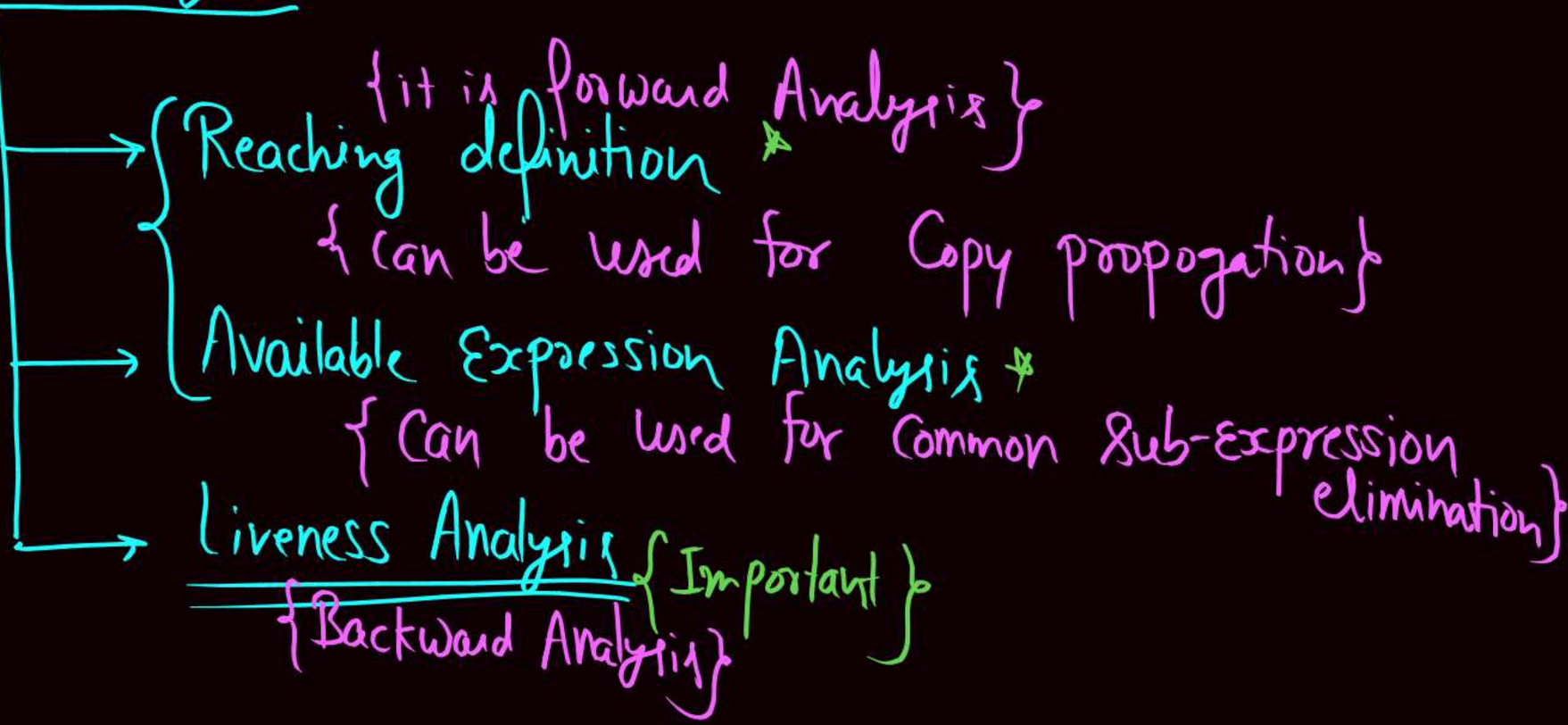
B.B = 2

Edges = 4

Slide

* Data flow analysis will be done using Control flow graph:-

* Data Flow Analysis





Topic : Liveness Analysis

- Live Variable:

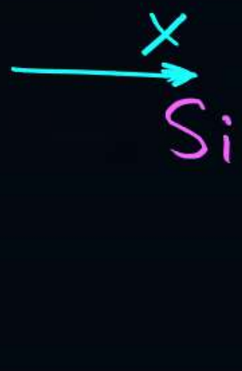
A variable x is live at a particular point in the program if it holds a value that can be used in the future.



Topic : Liveness Analysis

x is Live at statement S_i IFF → just before Si

1. There is a statement S_j that reads x
2. There is a path in b/w S_i and S_j
3. No write into x before S_j



y

① x = a + b
② y = x + a
③ z = b + y

Slide



Topic : Liveness Analysis

{ the value present in x just before the execution of statement S_i }

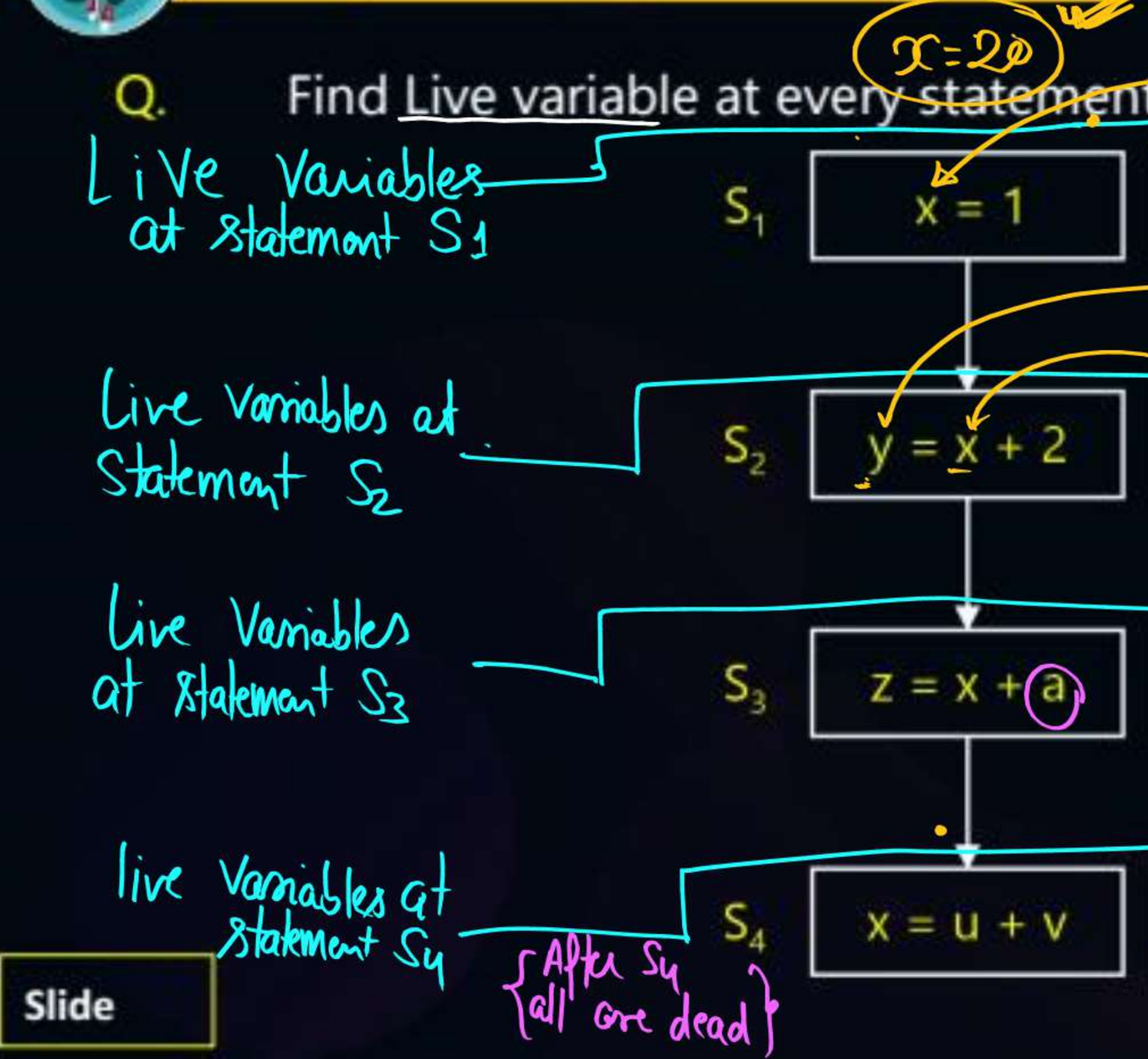
x is Live at statement S_i IFF

1. The variable x is live at the statement S_i if [redacted] could be used along some path in the flow graph, starting from S_i otherwise x is dead at S_i.
2. There should not be any assignment to x before it is used

Slide

Topic : Liveness Analysis

Q. Find Live variable at every statement.



Live Variables at Statement S_1

Live variables at Statement S_2

Live Variables at Statement S_3

live Variables at Statement S_4

{After S_4 all are dead}

no read of x before update by S_1

No read of y before update by S_2

x	y	a	z	u	v
dead	dead	live	dead	live	live
live	dead	live	dead	live	live
live	dead	live	dead	live	live
dead	dead	dead	dead	live	live

Slide

Topic : Liveness Analysis

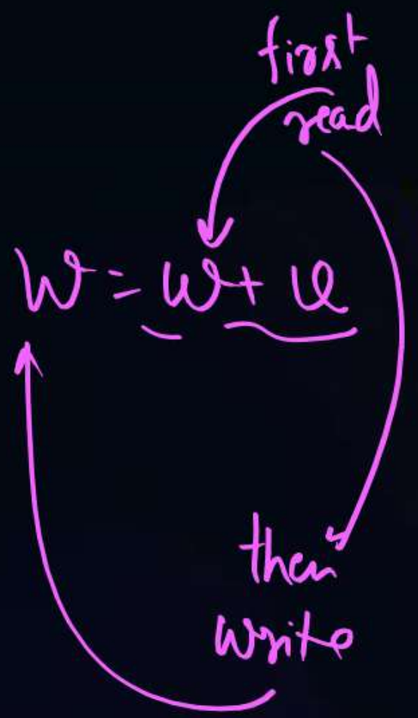
Q. Find Live variable at every statement.

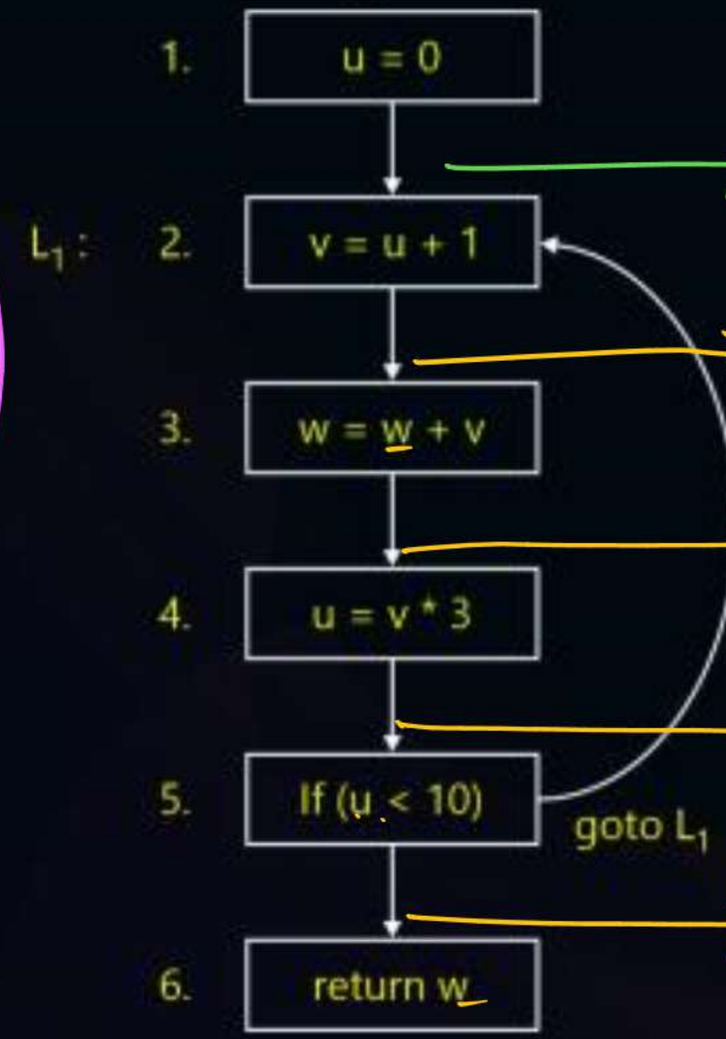
		v	z	x	y	w	u
	at S_1 →	✗	✗	✗	✗	✗	✗
1)	v = 1						
	at S_2 →	✓	✗	✗	✗	✗	✗
2)	z = v + 1						
	at S_3 →	✓	✓	✗	✗	✗	✗
3)	x = z * v						
	at S_4 →	✗	✓	✓	✗	✗	✗
4)	y = x * 2						
	at S_5 →	✗	✓	✓	✓	✗	✗
5)	w = x + z * y						
	at S_6 →	✗	✓	✗	✓	✓	✗
6)	u = z + 2						
	at S_7 →	✗	✗	✗	✓	✓	✓
7)	v = u + w + y						
	at S_8 →	✓	✗	✗	✗	✗	✓
8)	return v + u						
	after S_8 →	✗	✗	✗	✗	✗	✗

Slide

Topic : Liveness Analysis

Q. Find Live variable at every statement.





u v w

× × ✓

✓ × ✓

× ✓ ✓

× ✓ ✓

✓ × ✓

× × ✓

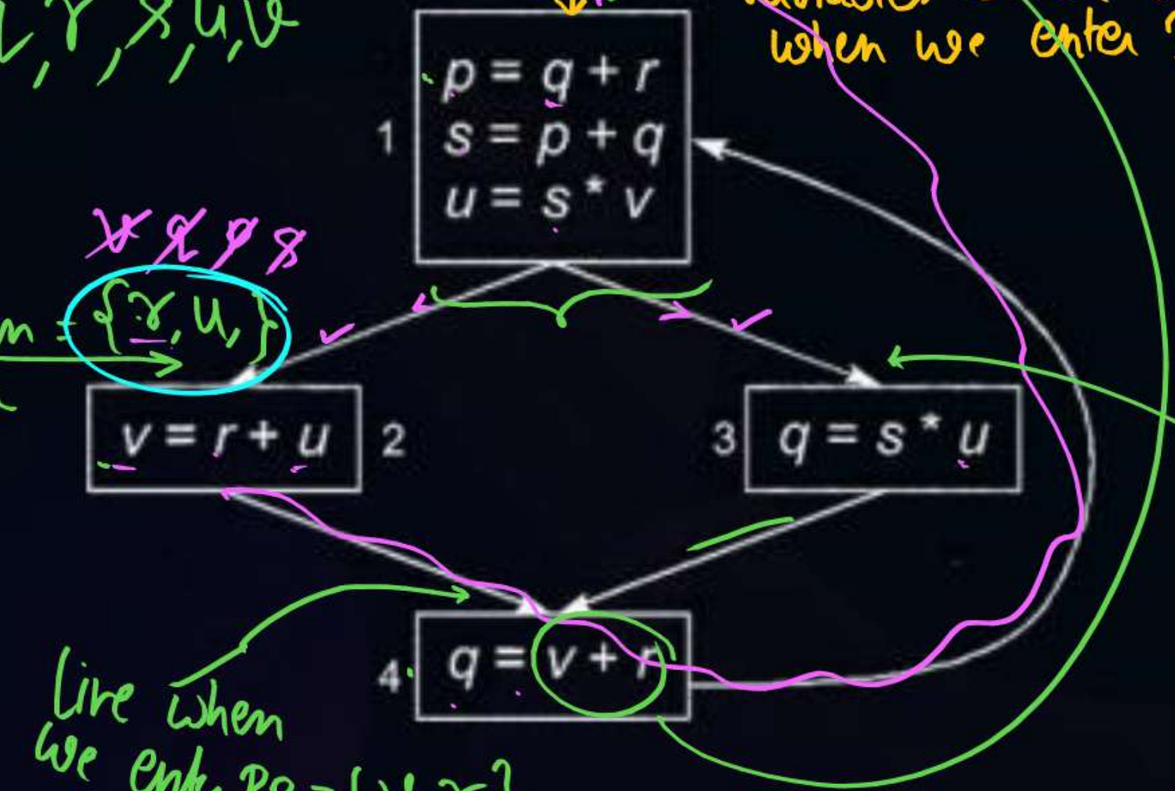
Slide

A variable x is said to be live at a statement S_i in a program if the following three conditions hold simultaneously:

1. There exists a statement S_j that uses x
2. There is a path from S_i to S_j in the flow graph corresponding to the program
3. The path has no intervening assignment to x including at S_i and S_j

p, q, r, s, u, v

Variables which are live when we enter B.B$_1$ = {q, r, v, p, s, u}



live when we enter BB$_2$ = {r, u}

Live when we enter BB$_2$ = {v, r}

The variables which are live both at the statement in basic block 2 and at the statement in basic block 3 of the above control flow graph are:

live when we enter BB$_3$ = {s, u, v, r}

Ans = {r, u}

Slide



Topic : Liveness Analysis at a node n

- A node n may be an individual statement or a basic block.
- Outgoing edges from node n leads to successor of n "Succ[n]"
- Liveness of variable flows around the edges of the graph.

1. GEN[n]/Use[n] = Set of variables used in node n, and there should be no definition of that variable in node n before its use.
2. KILL[n]/Def[n] = Set of variables defined by node n.
3. In[n] = Set of variables which are live at the starting of node n
4. Out[n] = Set of variables which are live at the end of node n.

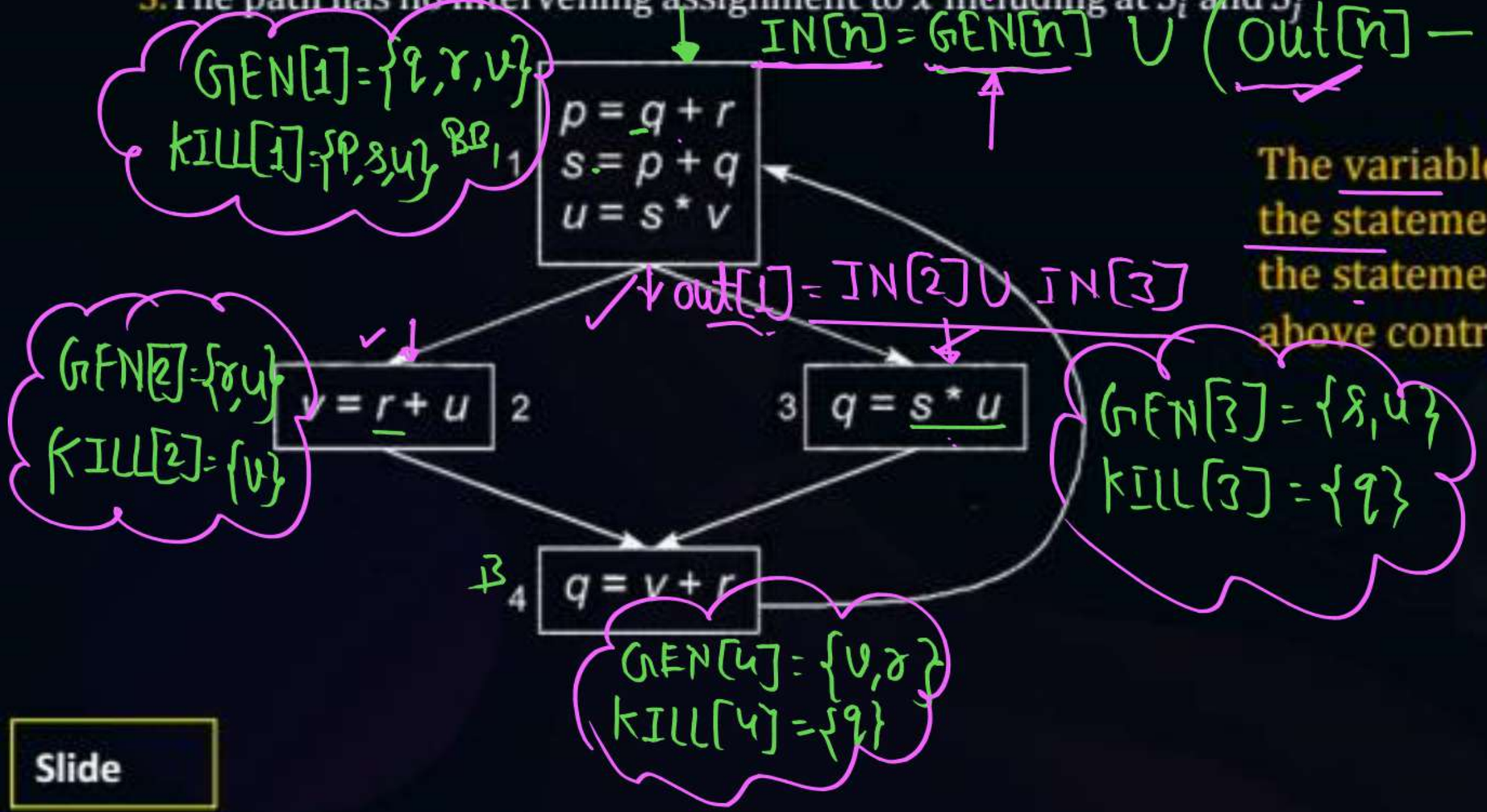
- In[n] = Use[n] ∪ (Out[n] – Def[n])
- Out[n] = ∪ In[S] , $\forall S \in Succ[n]$

Slide

A variable x is said to be live at a statement S_i in a program if the following three conditions hold simultaneously:

1. There exists a statement S_j that uses x
2. There is a path from S_i to S_j in the flow graph corresponding to the program
3. The path has no intervening assignment to x including at S_i and S_j

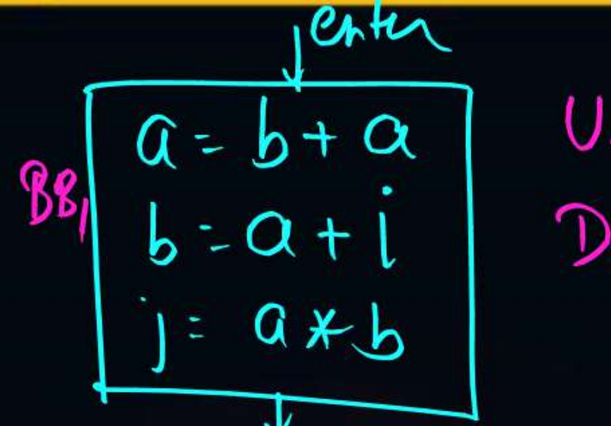
$IN[n] = GEN[n] \cup (OUT[n] - KILL[n])$



The variables which are live both at the statement in basic block 2 and at the statement in basic block 3 of the above control flow graph are:

Slide

Topic : Liveness Analysis at a node n



Use[1] = {b, i, a}
Def[1] = {a, b, j}

Use[2] = {a, b, c}
Def[2] = {x, y}

Use[3] = {z, x}
Def[3] = {z}

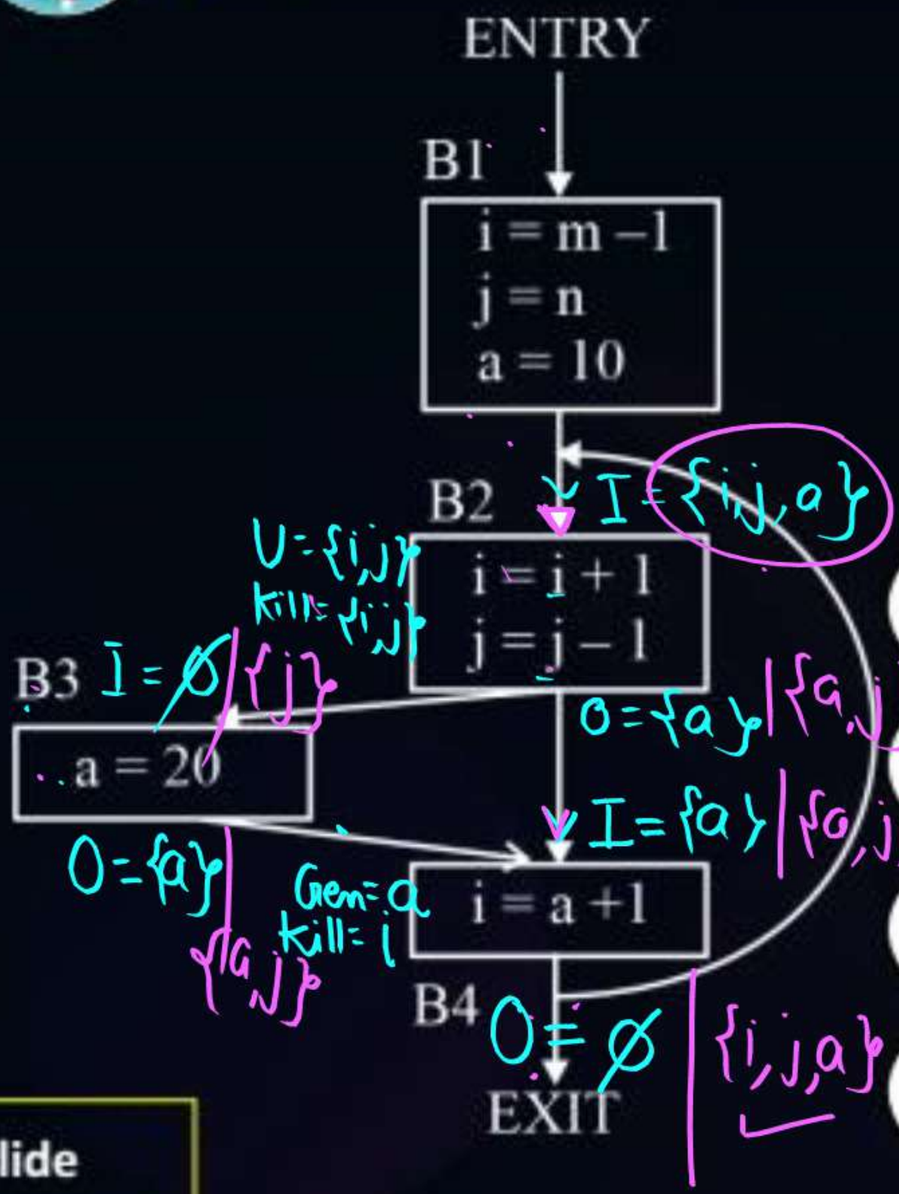
Use[4] = {z, y}
Def[4] = {a, b}

Block	Use	Def	Out	IN
BB_1	{b, i, a}	{a, b, j}	{a, b, c, z}	{b, i, a, c, z}
BB_2	{a, b, c}	{x, y}	{z, x, y}	{a, b, c, z}
BB_3	{z, x}	{z}	{z, y}	{z, x, y}
BB_4	{z, y}	{a, b}	∅	{z, y}

IN[4] = Use[4] ∪ (Out[4] − Def[4])

Slide

Topic : Liveness Analysis at a node n



Consider the control flow graph shown.

Which one of the following choices lists the set of live variable at the exit point of each basic block?

A. B1: { }, B2: {a}, B3: {a}, B4: {a}

B. B1: {i, j}, B2: {a}, B3: {a}, B4: {i}

C. B1: { a, i, j }, B2: {a, i , j}, B3: {a, i}, B4: {a}

D. B1: { a, i , j }, B2: {a, j}, B3: {a, j}, B4: {a, i, j}

Slide



Topic : Runtime Environment

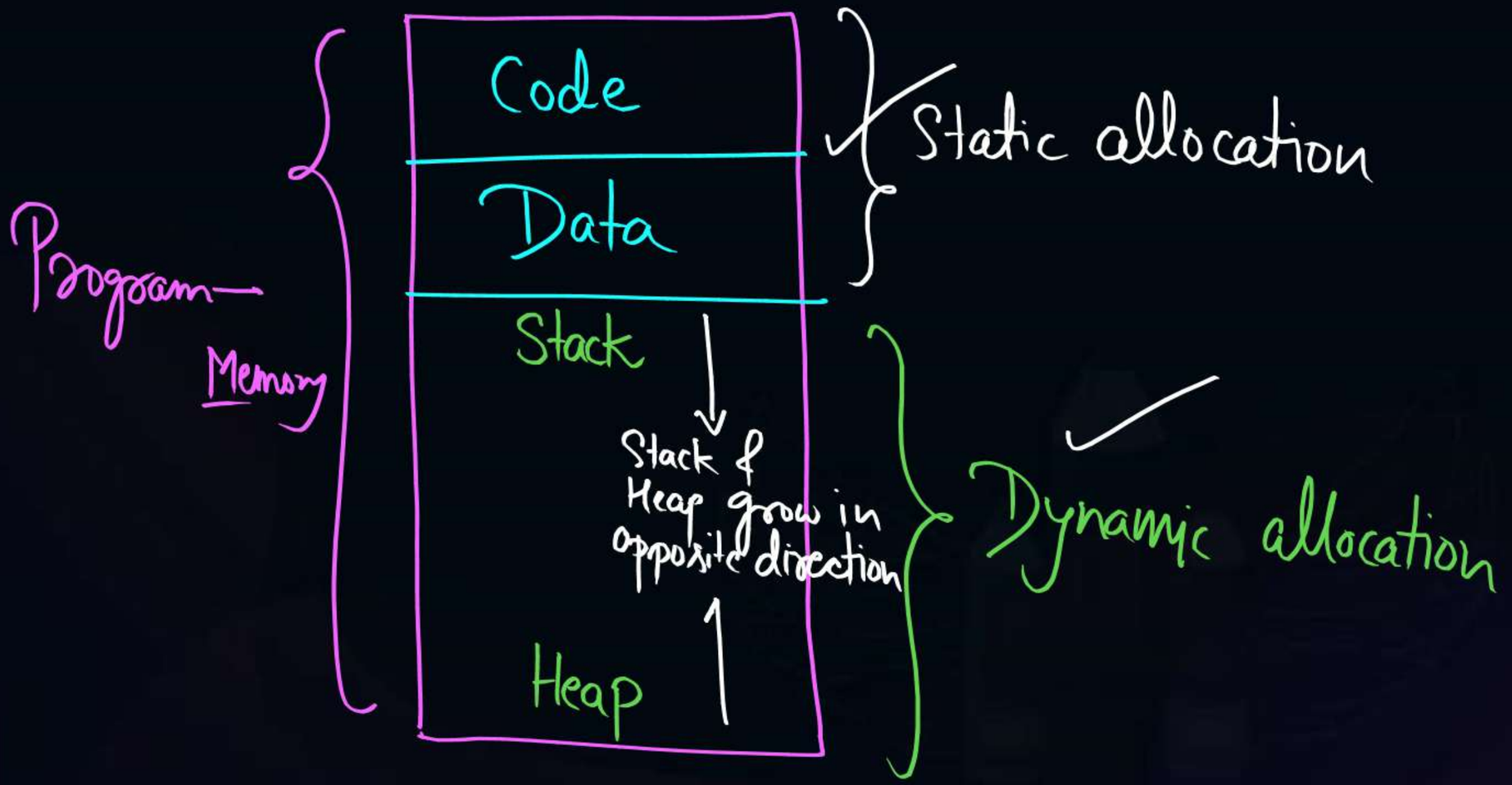
Runtime environment manages runtime memory requirements.

Different storage allocation strategies are used to manage this memory requirements.

1. Static Allocation
2. Stack Allocation
3. Heap Allocation



Topic : Sub-division of run-time memory



Slide

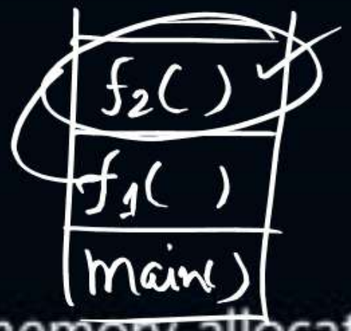


Topic : Static allocation

In this allocation scheme, the compiled data is bound to a fixed location in the memory and it does not change when the program executes. As the memory requirement and storage locations are known in advance, runtime support for memory allocation and de-allocation is not required.



Topic : Stack Allocation



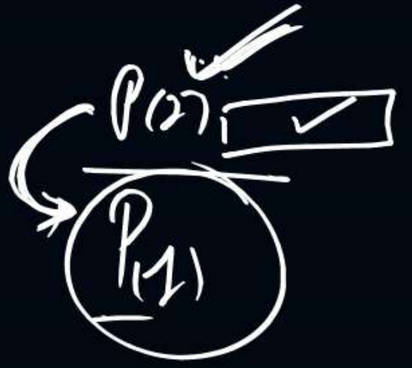
Procedure calls and their activations are managed by means of stack memory allocation. It works in last-in-first-out (LIFO) method and this allocation strategy is very useful for recursive procedure calls.

A procedure name is pushed on to the stack when it is called (activation begins) and it is popped when it returns (activation ends). Information needed during execution of a procedure is managed using an activation record.

When a procedure is called, an activation record is pushed into the stack and as soon as the control returns to the caller function the activation record is popped



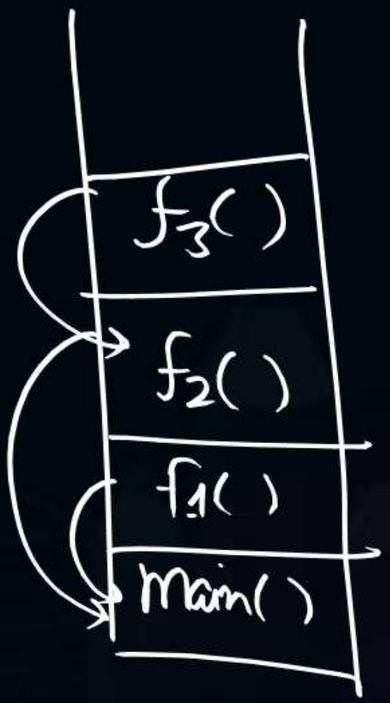
Topic : Activation Record



In general, activation record consists of the following things

- Temporaries
- Local Data
- Machine Status
- Control Link
- Access Link
- Return Value
- Actual Parameters

Read Room Book



Slide



Topic : Heap allocation

Variables local to a procedure are allocated and de-allocated only at runtime. Heap allocation is used to dynamically allocate memory to the variables and claim it back when the variables are no more required.



Topic : Miscellaneous

- There is no algorithm to check whether the given grammar is ambiguous or not and hence it is an undecidable problem.
- There is no algorithm to convert ambiguous grammar into unambiguous grammar hence it is an undecidable problem.
- Grammar from which we can not eliminate ambiguity is called inherently ambiguous grammar.



Topic : Miscellaneous

- If LR parser table is constructed using YACC tool, then YACC will resolve the Shift-Reduce conflict in favour of Shift. (No resolution for Reduce-Reduce)
- LR(1) parsing is sufficient for deterministic context-free languages.
- For any context-free grammar, there is a parser that takes at most $O(n^3)$ time to parse a string of length n.

it is CYK parsa Algo



2 mins Summary

Topic SSA

Topic Control Flow Graph

Topic Liveness Analysis

Slide


THANK - YOU
Slide